Crew Quickstart

This guide takes you from nothing installed to a first ticket worked in one repository. It assumes a machine with network access and a Git repository you can write to. Every step says how to check that it worked before you move on.

The Ten-Minute Checklist

#	Step	Check	Min
1	Install Claude Code and the marketplace	`claude --version` prints a version	3
2	Install the `crew` plugin	`/crew:status` answers	1
3	Set up the repository with `/crew:init`	`/crew:status` shows `config .crew/config.json schema 7`	3
4	Only a repository crew 0.20 or older set up: `/crew:migrate`	`/crew:status` no longer says `run /crew:migrate`	1
5	First ticket	a ticket directory under `.work/tickets/`	2

1. Install Claude Code and the Marketplace

Windows

Open PowerShell. Administrator is recommended, not required.

```
git clone git@github.com:mbadali25/useful-claude-add-ons.git

cd useful-claude-add-ons

.\scripts\install-prerequisites.ps1
```

Tick **This repo** in the menu. When the script finishes, open a **new** PowerShell window so the `PATH` change applies, then run `claude --version`.

Hooks run under PowerShell on Windows. `python3` is often missing from Git Bash: if a crew command reports it cannot find Python, install Python 3 and make sure `python` or `py -3`

works in the shell you use.

Linux

```
git clone git@github.com:mbadali25/useful-claude-add-ons.git

cd useful-claude-add-ons

./scripts/install-prerequisites.sh
```

Tick **This repo** in the menu. Run `source ~/.bashrc` (or open a new shell), then `claude --version`.

Already Have Claude Code?

Skip the scripts and add the marketplace directly:

```
claude plugin marketplace add mbadali25/useful-claude-add-ons
```

2. Install the Crew Plugin

```
claude plugin install crew@useful-claude-add-ons
```

crew registers hooks, so it is off in the install menu by default. Start `claude` in your repository and run `/crew:status`. Before step 3 it reads `config none - run /crew:init`. That is the expected answer.

3. Set up the Repository

```
/crew:init
```

Init is phased and resumable: it detects the platform, writes the config, and asks before each change. Stop after the config phase if you are short on time; `/crew:init` picks up where it left off.

Init writes the repository's crew config to `.crew/config.json`. That is the one file crew reads its repo settings from: `/crew:config` edits it, every gate and guard reads it, and nothing needs migrating afterwards.

Check: `/crew:status` now shows `config .crew/config.json schema 7` with no `run /crew:migrate` after it, and a `gitignore current` line: init added the ignore patterns for the languages it found, inside one `# crew:gitignore:managed` block at the top of `.gitignore`, without touching your own lines. A new repository skips step 4.

4. Repositories Crew 0.20 Set up: Migrate Once

Only when `/crew:status` ends its `config` line with `run /crew:migrate`: the repository still holds a crew 0.20 (or older) setup - a role 1.0 removed, a schema older than 7, ticket files in `.work/tickets/`, or a PM journal. Then:

```
/crew:migrate
```

It previews first and writes nothing until you agree. The preview lists:

- the files it will create (`.crew/crew.json`, a schema-1 record of the old config; `.crew/metrics.jsonl`; one directory per ticket under `.work/tickets/`);
- any config key it did not recognise, which is kept under `unmapped`, never dropped;
- the originals it leaves in place. Keep `.crew/config.json` even though the preview calls it `retireable`: crew still reads its settings from that file, and `crew.json` does not replace it.

Say yes, and it backs up to `.crew/backups/migrate-<time>/`, then applies. The line it ends with is your undo:

```
/crew:migrate --rollback .crew/backups/migrate-<time>
```

Your code map in `.crew/codemap/` and its anchors are not touched.

Check: `/crew:status` shows `config .crew/crew.json schema 1`.

5. Your First Ticket

In crew 1.0 one interactive session owns a ticket from start to finish: brainstorm, spec, plan, implement, tests, docs, review, done.

For a small first change use the light path:

```
/crew:fix <one sentence: what is wrong and where>
```

The light path runs every phase in short form: a one-line direction, a short spec, a one-step plan, and your approval before any edit. Review runs after tests and docs, and has a budget of three rounds (counted across every plan the ticket runs under).

Check: `.work/tickets/<id>/` exists and `/crew:status` lists it under `open`.

First UI Ticket: `/crew:webtest`

If your first ticket touches a web UI, `/crew:init`'s Phase 6 (Browser tests) installs Playwright and asks for two or three specs where breakage is expensive, plus visual baselines for the pages that matter — do that once, during setup, not per ticket. From then on, work the ticket normally and type `/crew:webtest` where you would otherwise hand-write the spec file: it runs the planner -> generator -> healer loop against the ticket's acceptance criteria and hands the reviewer a trace and an accessibility report alongside the diff. See Daily workflow for the phase-by-phase detail.

Check: a `specs/<ticket>.md` file exists and `npx playwright test` passes with no agent attached.

Optional: Let a Long Session Wrap up, Clear and Resume Itself

When the context fills, crew asks the session once to finish or park its work and write `.work/HANDOFF.md`. The next session starts with that handoff and its next action. Both steps are on by default.

Auto-clear sits between those two steps: it types `/clear` for you. It is **off** by default and is turned on per machine, in `~/.claude/crew/config.json` (Windows: `%USERPROFILE%\.claude\crew\config.json`):

```
{ "context": { "autoClear": { "enabled": true } } }
```

To turn it off again, set `context.autoClear.enabled` to `false` in the same file. To turn it off in one repository, set it to `false` in that repository's `.crew/config.json`; setting it to `true` there does not turn it on. See `auto-cycle.md` for the full key list and for what each line in `.crew/.autoclear.log` means.

Check: after a wrap-up, `.crew/.autoclear.log` has a `sent` line or a `refusing - <reason>` line.

If Something Goes Wrong

Symptom	Do
`/crew:status` is not a known command	`claude plugin update crew@useful-claude-add-ons`, then restart `claude`
`migrate` reports a `CONFLICT`	read the path it names; apply refuses until it is resolved
`migrate` reports an interrupted apply	`/crew:migrate --rollback <dir>` first
`migrate could not tell - <path>: <why>`	fix or remove the named entry under `.crew/backups/`; the rest of the report is unaffected
a crew command cannot find Python	install Python 3; on Windows make `python` or `py -3` work

See Troubleshooting for the rest.